

Central de Missões

Contexto

Você deverá desenvolver um aplicativo chamado **Central de Missões** utilizando **Flutter + Firebase Cloud Firestore**.

O aplicativo será usado para cadastrar pequenas missões ou metas pessoais, por exemplo:

- Estudar Flutter por 30 minutos
- Fazer exercício
- Ler 10 páginas
- Organizar o quarto
- Estudar para uma prova

Cada missão deverá possuir um **título**, uma **dificuldade**, uma quantidade de **pontos** e um **status** indicando se ela foi concluída.

Objetivo

O aplicativo deverá permitir:

```
None
CADASTRAR MISSÃO
    ↓
SALVAR NO FIRESTORE
    ↓
LISTAR MISSÕES
    ↓
CONCLUIR OU EXCLUIR
    ↓
SOMAR OS PONTOS DAS MISSÕES CONCLUÍDAS
```

Todas as alterações realizadas no aplicativo deverão também acontecer no **Firebase**.

Firebase

No Cloud Firestore, utilize uma collection chamada:

```
None  
missoes
```

Cada documento deverá possuir os seguintes campos:

```
None  
titulo  
dificuldade  
pontos  
concluida
```

Exemplo:

```
None  
titulo: "Estudar Firebase"  
dificuldade: "Difícil"  
pontos: 30  
concluida: false
```

A estrutura deverá ficar semelhante a:

```
None  
missoes  
|  
├── documento1  
|   ├── titulo: "Estudar Firebase"  
|   ├── dificuldade: "Difícil"  
|   ├── pontos: 30  
|   └── concluida: false  
|  
└── documento2  
    ├── titulo: "Caminhar"  
    ├── dificuldade: "Fácil"  
    ├── pontos: 10  
    └── concluida: true
```

Sistema de pontos

Cada dificuldade terá uma quantidade de pontos:

None

Fácil → 10 pontos

Médio → 20 pontos

Difícil → 30 pontos

Ao cadastrar uma missão, o aplicativo deverá definir os pontos de acordo com a dificuldade escolhida.

Exemplo:

None

Título:

Estudar Firebase

Dificuldade:

Difícil

Pontos:

30

O valor também deverá ser salvo no Firestore.

Funcionalidades obrigatórias

1. Cadastrar uma missão

O usuário deverá informar:

None

Título da missão

Dificuldade

Exemplo:

None

Título:

Estudar Firebase

Dificuldade:

Difícil

[CADASTRAR MISSÃO]

O aplicativo deverá definir automaticamente os pontos:

None

Fácil → 10

Médio → 20

Difícil → 30

Ao cadastrar, a missão deverá ser salva no Firestore com:

None

titulo

dificuldade

pontos

concluida: false

2. Listar as missões

A tela principal deverá mostrar todas as missões cadastradas no Firebase.

Exemplo:

None

CENTRAL DE MISSÕES

Estudar Firebase

Dificuldade: ★★ ★

Pontos: 30

Status: Pendente

[CONCLUIR] [EXCLUIR]

Caminhar

Dificuldade: ★

Pontos: 10

Status: Concluída

[EXCLUIR]

3. Concluir uma missão

Ao pressionar:

None

CONCLUIR

o campo:

None

concluida

deverá mudar de:

None

false

para:

None

true

Essa alteração também deverá acontecer no documento correspondente no **Firestore**.

Depois da atualização, a tela deverá mostrar:

None

Status: Concluída

Também deverá ser exibida uma mensagem informando a quantidade de pontos conquistados.

Exemplo:

None

Missão concluída!

Você conquistou 30 pontos.

4. Pontuação total

O aplicativo deverá mostrar a quantidade total de pontos conquistados.

Somente as missões com:

None

concluida: true

deverão contar na pontuação.

Exemplo:

None

Missão 1

10 pontos

Concluída

Missão 2

20 pontos

Concluída

Missão 3

30 pontos

Pendente

Pontuação exibida:

None

PONTOS CONQUISTADOS: 30

A missão pendente não entra na soma.

5. Excluir uma missão

Cada missão deverá possuir a opção:

None

EXCLUIR

Ao excluir:

None

Missão removida da tela

+

Documento removido do Firestore

Caso uma missão concluída seja excluída, sua pontuação também não deverá mais aparecer no total.

Arquitetura do projeto

Utilize a organização trabalhada em aula:

None

Page

↓

Provider

↓

Service

↓

Firebase

Estrutura sugerida:

None

```
lib/  
|  
├─ main.dart  
├─ firebase_options.dart  
|  
├─ models/  
|   └─ missao.dart  
|  
├─ providers/  
|   └─ missao_provider.dart  
|  
├─ services/  
|   └─ missao_service.dart  
|  
└─ pages/  
    └─ home_page.dart
```

Model

Crie a classe Missao.

Você pode utilizar esta estrutura como base:

None

```
class Missao {  
  String? id;  
  String titulo;  
  String dificuldade;  
  int pontos;  
  bool concluida;  
  
  Missao({  
    this.id,  
    required this.titulo,  
    required this.dificuldade,  
    required this.pontos,  
    this.concluida = false,  
  });  
}
```

```
});  
}
```

Implemente também:

```
None  
toMap()  
fromMap()
```

Esses métodos serão utilizados para converter os dados entre:

```
None  
Objeto Dart  
  ↑  
Firebase
```

Service

A `MissaoService` deverá ser responsável pela comunicação com o Firebase.

Ela deverá possuir métodos para:

```
None  
adicionar  
buscarTodas  
atualizar  
excluir
```

Utilize os comandos do Firestore trabalhados em aula:

```
None  
.add()  
.get()  
.update()  
.delete()
```

Provider

O MissaoProvider deverá fazer a ligação entre a tela e o Service.

Fluxo esperado:

```
None
HomePage
  ↓
MissaoProvider
  ↓
MissaoService
  ↓
Cloud Firestore
```

Evite acessar o Firebase diretamente dentro da HomePage.

O Provider também poderá ser responsável por calcular a pontuação total das missões concluídas.

Representação da dificuldade

A dificuldade pode ser representada por estrelas:

```
None
Fácil
★
10 pontos

Médio
★★
20 pontos

Difícil
★★★
30 pontos
```

Não é necessário mudar cores nem criar um visual complexo.

Desafio extra 1 — Data

Adicione o campo:

None
data

Exemplo:

None
título: "Estudar Firebase"
dificuldade: "Difícil"
pontos: 30
concluida: false
data: "23/09/2026"

A data também deverá aparecer na tela.

Desafio extra 2 — Tela de detalhes

Ao clicar em uma missão, abra uma nova tela.

Exemplo:

None
DETALHES DA MISSÃO

Título:
Estudar Firebase

Dificuldade:
★★★★ Difícil

Pontos:
30

Status:
Pendente

[CONCLUIR MISSÃO]

Resultado esperado

Ao final da atividade, o aplicativo deverá possuir o seguinte fluxo:

```
None
USUÁRIO CADASTRA UMA MISSÃO
    ↓
APP DEFINE OS PONTOS
    ↓
FIREBASE SALVA A MISSÃO
    ↓
APP LISTA AS MISSÕES
    ↓
USUÁRIO CONCLUI UMA MISSÃO
    ↓
FIREBASE É ATUALIZADO
    ↓
PONTUAÇÃO TOTAL É ATUALIZADA
```

O principal objetivo desta atividade é praticar a integração entre:

```
None
Flutter
+
Provider
+
Service
+
Firebase Cloud Firestore
```

e também trabalhar operações de:

None

CREATE

READ

UPDATE

DELETE